

Dokumentacja wdrożeniowa

Plan realizacji krok po kroku do złożenia kompletnego produktu

1. Cel dokumentu

Celem tego dokumentu jest zdefiniowanie **praktycznej, wykonawczej ścieżki realizacji projektu** od etapu koncepcji technicznej do etapu złożenia w jedną działającą całość:

- aplikacji webowej,
- backendu,
- lokalnego silnika audio,
- protokołu komunikacji,
- onboarding flow,
- wersji MVP gotowej do testów z użytkownikami.

Dokument nie opisuje wizji produktu w ujęciu marketingowym. Skupia się na **kolejności prac, zależnościach, kryteriach gotowości oraz technicznej logice realizacji**.

2. Zasady realizacji

Przed rozpoczęciem implementacji należy przyjąć pięć zasad, które będą obowiązywały przez cały projekt.

2.1 Zasada 1 — najpierw rdzeń, potem warstwa produktowa

Najpierw należy udowodnić, że działa rdzeń techniczny sesji realtime: - wejście audio, - detekcja, - timing, - scoring, - komunikacja z webem.

Dopiero po tym należy rozwijać: - dashboard, - rankingi, - płatności, - rozbudowane UI.

2.2 Zasada 2 — engine jest źródłem prawdy

Wszystkie decyzje dotyczące: - wykrytej nuty, - czasu zagrania, - oceny trafienia, - końcowego wyniku muszą być liczone przez local engine.

2.3 Zasada 3 — frontend nie może blokować rdzenia

Jeżeli trzeba wybierać pomiędzy: - dopracowaniem interfejsu, - a działającą komunikacją i scoringiem, zawsze priorytet ma scoring, synchronizacja i stabilność sesji.

2.4 Zasada 4 — każdy etap musi kończyć się działającym artefaktem

Każda faza musi kończyć się czymś mierzalnym i uruchamialnym. Nie wolno prowadzić kilku niedokończonych linii rozwoju jednocześnie.

2.5 Zasada 5 — najpierw monofonia i prosty use case

Pierwsza kompletna wersja ma obsługiwać: - pojedyncze dźwięki, - prosty trening, - jedną ścieżkę rozgrywki, - jedną metodę oceny.

Nie należy zaczynać od: - akordów, - wielu trybów gry, - zaawansowanej analizy technik gitarowych, - multiplayera.

3. Definicja końcowego stanu docelowego

Projekt można uznać za złożony w kompletną całość, gdy spełnione są wszystkie poniższe warunki:

1. użytkownik może założyć konto,
2. użytkownik może pobrać i uruchomić local engine,
3. strona wykrywa uruchomiony engine,
4. użytkownik może przejść prostą kalibrację,
5. użytkownik może wybrać trening,
6. trening uruchamia się poprawnie,
7. engine przechwytyuje sygnał z gitary,
8. engine wykrywa poprawne i niepoprawne zagrania,
9. frontend pokazuje wynik w czasie rzeczywistym,
10. wynik sesji zapisuje się w backendzie,
11. progres użytkownika zostaje zaktualizowany,
12. cały flow jest stabilny na poziomie pozwalającym na testy z użytkownikami.

To jest punkt końcowy dla wersji MVP złożonej w całość.

4. Faza 0 — przygotowanie wykonawcze

4.1 Cel fazy

Ustalenie niezbędnych fundamentów przed rozpoczęciem kodowania właściwego systemu.

4.2 Zakres

Na tym etapie należy przygotować: - główną dokumentację architektoniczną, - decyzje technologiczne, - strukturę repozytorium, - zasady wersjonowania, - podstawowy model domenowy, - listę milestone'ów, - definicję MVP.

4.3 Decyzje, które muszą zapaść przed implementacją

A. Stack webowy

Rekomendacja: - Next.js - TypeScript - Tailwind CSS

B. Backend

Rekomendacja startowa: - Supabase lub PostgreSQL + własne API

C. Local engine

Rekomendacja: - Rust lub C++ - komunikacja przez WebSocket na localhost

D. Repozytorium

Rekomendacja: - monorepo

4.4 Deliverables fazy 0

- opis architektury systemu,
- backlog MVP,
- model modułów,
- wstępna specyfikacja protokołu,
- plan kolejności implementacji.

4.5 Kryterium zakończenia fazy 0

Faza 0 jest zakończona, gdy zespół lub wykonawca jest w stanie odpowiedzieć bez wahania: - co budujemy najpierw, - co jest poza MVP, - który komponent jest źródłem prawdy, - jak komponenty będą się komunikować, - jaki jest pierwszy techniczny milestone.

5. Faza 1 — zbudowanie technicznego Proof of Concept rdzenia

To jest najważniejsza faza projektu. Jeżeli ta faza się nie powiedzie, nie ma sensu rozwijać warstwy produktowej.

5.1 Cel fazy

Udowodnić, że można: - przechwycić sygnał z gitary, - wykryć podstawową nutę, - wykryć moment ataku, - przesłać wynik do warstwy webowej, - zobaczyć ten wynik w prostym UI.

5.2 Zakres fazy

Faza obejmuje wyłącznie: - local engine, - minimalny frontend testowy, - komunikację lokalną.

Bez: - logowania, - dashboardu, - zapisów do bazy, - systemu progresji.

5.3 Kolejność prac w fazie 1

Krok 1 — utworzenie repo i podstawowej struktury projektu

Należy przygotować strukturę monorepo z minimum: - `apps/web` - `apps/engine` - `packages/protocol` - `packages/shared-types` - `docs`

Krok 2 — uruchomienie local engine jako samodzielnego procesu

Pierwszym celem engine nie jest jeszcze scoring, tylko stabilne uruchomienie procesu lokalnego i podstawowy runtime.

Wymagania: - proces startuje lokalnie, - ma podstawowy logger, - potrafi zgłosić status „ready”, - potrafi się zamknąć bez błędów.

Krok 3 — implementacja wykrywania urządzeń audio

Engine musi: - pobrać listę urządzeń wejściowych, - wskazać aktualne urządzenie, - zwrócić podstawowe parametry urządzenia, - pozwolić na wybór urządzenia wejściowego.

Krok 4 — implementacja audio capture

Po wyborze urządzenia engine musi: - uruchomić wejście audio, - pobierać próbki, - utrzymywać stabilny bufor, - logować podstawowe metryki.

Rezultat tego kroku: - można włączyć wejście z interfejsu, - można potwierdzić, że sygnał płynie.

Krok 5 — implementacja preprocessing pipeline

Należy dodać minimalną obróbkę sygnału: - high-pass filter, - noise gate, - poziom sygnału, - podstawową normalizację.

Celem nie jest doskonałe DSP, tylko uzyskanie stabilniejszego sygnału wejściowego.

Krok 6 — implementacja onset detection

Należy wykrywać moment rozpoczęcia dźwięku.

Wymagany wynik: - engine generuje event onsetu, - event ma timestamp wewnętrzny, - fałszywe wykrycia są ograniczone do akceptowalnego poziomu.

Krok 7 — implementacja pitch detection dla pojedynczych nut

Na tym etapie należy wdrożyć pierwszą wersję detekcji wysokości dźwięku.

Wymagania: - detekcja pojedynczej nuty, - zwracanie częstotliwości w Hz, - mapowanie do nazwy nuty, - confidence score, - odrzucanie zbyt niepewnych wyników.

Krok 8 — zbudowanie event modelu po stronie engine

Engine powinien generować ustandaryzowane eventy, np.: - `note_detected` - `engine_ready` - `device_list` - `error`

To jest moment na ustalenie pierwszej wersji protokołu komunikacyjnego.

Krok 9 — implementacja lokalnego WebSocket servera

Engine uruchamia lokalny serwer WebSocket i potrafi: - przyjąć połączenie z weba, - wysłać status, - streamować eventy nut.

Krok 10 — przygotowanie prostego webowego debug klienta

Na stronie należy stworzyć bardzo prosty ekran, który: - łączy się z engine, - pokazuje status połączenia, - wyświetla wykrywane nuty, - wyświetla confidence i timestamp.

To nie ma być jeszcze gameplay UI, tylko narzędzie walidacyjne.

5.4 Deliverables fazy 1

- engine uruchamiany lokalnie,
- działające audio capture,
- podstawowy onset detection,
- podstawowy pitch detection,
- lokalny WebSocket,
- webowy debug panel z live eventami.

5.5 Kryterium zakończenia fazy 1

Faza 1 jest zakończona, gdy użytkownik może: - uruchomić engine, - podłączyć gitarę, - wejść na stronę, - zobaczyć wykrywane nuty w czasie rzeczywistym.

Jeżeli ten warunek nie jest spełniony, nie wolno przechodzić do fazy 2.

6. Faza 2 — zbudowanie kompletnej sesji treningowej offline względem backendu

6.1 Cel fazy

Złożyć pierwszy kompletny przepływ sesji treningowej bez uzależniania go jeszcze od całego systemu produktowego.

W tej fazie trzeba udowodnić, że działa cała pętla: - uruchom sesję, - podaj mapę nut, - graj, - licz wynik, - zakończ sesję, - pokaż rezultat.

6.2 Zakres fazy

Na tym etapie można nadal pracować bez pełnego auth i bez pełnego backendu, o ile lokalnie da się zasymulować dane sesji.

6.3 Kolejność prac w fazie 2

Krok 1 — zdefiniowanie modelu treningu

Należy zdefiniować minimalny model treningu: - id sesji, - BPM, - typ ćwiczenia, - lista oczekiwanych eventów, - okna trafienia, - czas trwania.

Krok 2 — zdefiniowanie note map format

Każde ćwiczenie musi posiadać formalny zapis wydarzeń do zagrania.

Minimalna struktura jednego eventu: - timestamp, - expected note, - allowed hit window, - opcjonalna trudność lub metadata.

Krok 3 — implementacja runtime sesji po stronie engine

Engine musi umieć: - przyjąć konfigurację sesji, - zainicjalizować clock, - wczytać note map, - rozpocząć nasłuch, - porównać wykryte eventy do oczekiwanych eventów.

Krok 4 — implementacja timing evaluation

Silnik powinien wyznaczać: - czy nuta była poprawna, - jaki był offset czasowy, - czy mieści się w oknie trafienia.

Krok 5 — implementacja scoringu

Należy stworzyć prosty, jawny system oceny.

Minimalnie: - Perfect, - Good, - Miss, - combo, - accuracy, - session score.

Krok 6 — implementacja webowego ekranu sesji

Pierwszy ekran sesji powinien pokazywać: - countdown, - informację o aktualnym stanie sesji, - prostą oś czasu lub listę nut, - wynik hit/miss, - combo, - postęp.

Na tym etapie design może być prosty, ale logika musi być poprawna.

Krok 7 — implementacja ekranu końcowego

Po zakończeniu sesji użytkownik powinien zobaczyć: - accuracy, - score, - liczby trafień, - liczbę błędów, - czas sesji.

6.4 Deliverables fazy 2

- działający start sesji,

- działający runtime sesji,
- scoring po stronie engine,
- ekran treningu,
- ekran wyniku.

6.5 Kryterium zakończenia fazy 2

Faza 2 jest zakończona, gdy można przejść kompletną sesję od startu do wyniku bez udziału backendowej logiki produktu.

7. Faza 3 — zbudowanie stabilnej warstwy backendowej i produktowej

7.1 Cel fazy

Dołożyć warstwę kont użytkowników, zapisu wyników i podstawowego systemu progresji.

7.2 Zakres fazy

Ta faza obejmuje: - auth, - profile, - zapisywanie sesji, - progres, - content retrieval, - podstawowy dashboard.

7.3 Kolejność prac w fazie 3

Krok 1 — model danych

Należy zdefiniować schemat danych dla minimum: - user, - profile, - exercise, - session_template, - session_result, - progress_state.

Krok 2 — auth i session management

Należy wdrożyć: - rejestrację, - logowanie, - utrzymanie sesji użytkownika, - podstawową ochronę zasobów.

Krok 3 — content fetching

Aplikacja webowa musi umieć pobrać z backendu: - listę treningów, - szczegóły treningu, - mapę nut, - meta dane treningu.

Krok 4 — zapisywanie wyników

Po zakończonej sesji frontend lub backend musi zapisać: - wynik końcowy, - szczegóły sesji, - engine version, - accuracy, - czas, - podstawowe statystyki.

Krok 5 — aktualizacja progresu

Po zapisaniu wyniku system musi: - dodać XP, - zaktualizować streak lub aktywność, - odblokować kolejne elementy ścieżki, jeśli warunki są spełnione.

Krok 6 — dashboard MVP

Dashboard ma pokazywać tylko to, co potrzebne do pierwszej wersji produktu: - ostatnie sesję, - dzisiejszy cel, - poziom, - rekomendowany kolejny trening.

7.4 Deliverables fazy 3

- auth,
- profile,
- content source,
- zapis wyników,
- progres,
- prosty dashboard.

7.5 Kryterium zakończenia fazy 3

Faza 3 jest zakończona, gdy zalogowany użytkownik może: - wybrać trening, - przejść sesję, - otrzymać wynik, - zobaczyć zapisane efekty swojej pracy w dashboardzie.

8. Faza 4 — stabilizacja połączenia web ↔ engine i obsługa realnych warunków

8.1 Cel fazy

Wyliminować wszystkie problemy integracyjne, które pojawiają się poza idealnym scenariuszem.

8.2 Obszary obowiązkowe

A. Engine discovery

Web musi umieć wykryć, czy engine działa.

Scenariusze do obsłużenia: - engine uruchomiony, - engine nieuruchomiony, - engine uruchomiony, ale niedostępny, - niezgodna wersja engine.

B. Reconnect flow

Należy obsłużyć utratę połączenia podczas sesji i poza sesją.

C. Device state changes

Należy obsłużyć: - zmianę urządzenia audio, - odłączenie interfejsu, - brak sygnału, - zbyt niski poziom wejścia.

D. Błędy runtime

Należy przygotować jawne stany błędów i komunikaty dla użytkownika.

E. Version compatibility

Frontend musi wiedzieć, kiedy wymaga nowszej wersji engine.

8.3 Deliverables fazy 4

- status połączenia,
- reconnect strategy,
- obsługa braku engine,
- obsługa błędów urządzeń,
- obsługa niekompatybilności wersji.

8.4 Kryterium zakończenia fazy 4

Faza 4 jest zakończona, gdy system zachowuje się przewidywalnie również wtedy, gdy coś pójdzie nie tak.

9. Faza 5 — onboarding, kalibracja i przygotowanie do realnych użytkowników

9.1 Cel fazy

Zmniejszyć barierę wejścia i upewnić się, że nowy użytkownik potrafi uruchomić system bez pomocy developera.

9.2 Kolejność prac w fazie 5

Krok 1 — ekran wykrywania engine

Strona po wejściu do sekcji treningu powinna jasno pokazywać: - czy engine jest aktywny, - czy potrzebna jest instalacja, - czy potrzebna jest aktualizacja.

Krok 2 — prosty install flow

Należy przygotować: - ekran pobrania, - instrukcję uruchomienia, - potwierdzenie, że engine został wykryty.

Krok 3 — wybór urządzenia wejściowego

Użytkownik musi mieć prosty ekran do: - wyboru urządzenia, - sprawdzenia poziomu wejścia, - potwierdzenia aktywnego sygnału.

Krok 4 — kalibracja

Minimalna kalibracja powinna obejmować: - test poziomu wejścia, - test prostego zagrania, - ewentualne ustawienie progu czułości, - podstawową kompensację opóźnienia, jeśli zostanie wdrożona.

Krok 5 — pierwsza sesja onboardingowa

Pierwszy trening użytkownika powinien być bardzo prosty i służyć jednocześnie jako test całego flow.

9.3 Deliverables fazy 5

- install flow,
- detection flow,
- device setup,
- podstawowa kalibracja,
- pierwszy trening onboardingowy.

9.4 Kryterium zakończenia fazy 5

Faza 5 jest zakończona, gdy nowy użytkownik może samodzielnie przejść od zera do pierwszego ukończonego treningu.

10. Faza 6 — przygotowanie wersji MVP do testów zamkniętych

10.1 Cel fazy

Doprowadzić projekt do stanu, w którym można oddać go pierwszym testerom bez ciągłej interwencji technicznej.

10.2 Zakres

- poprawki UX,
- poprawki stabilności,
- logowanie błędów,
- telemetria,
- podstawowe QA end-to-end,
- przygotowanie checklist release.

10.3 Obowiązkowe elementy przed testami

- brak krytycznych crashy,
- przewidywalne stany błędów,
- zapis sesji działa,
- engine ma rozpoznawalną wersję,
- użytkownik rozumie, co ma zrobić, gdy coś nie działa.

10.4 Deliverables fazy 6

- build MVP,
- release notes,
- known issues list,
- telemetry basic,
- test plan.

10.5 Kryterium zakończenia fazy 6

Faza 6 jest zakończona, gdy produkt może zostać przekazany do zamkniętych testów bez obecności developera przy każdym uruchomieniu.

11. Kolejność implementacji modułów

Poniżej znajduje się zalecana kolejność implementacji. Nie należy jej odwracać bez ważnego powodu.

Priorytet P0

1. runtime engine
2. audio device detection
3. audio capture
4. preprocessing
5. onset detection
6. pitch detection
7. local WebSocket
8. debug web client
9. session runtime
10. scoring

Priorytet P1

1. gameplay screen
2. results screen
3. content model
4. backend storage
5. auth
6. progress system
7. dashboard

Priorytet P2

1. calibration UX
2. reconnect flow
3. version compatibility
4. onboarding flow
5. telemetry
6. richer analytics

Priorytet P3

1. leaderboard
 2. weekly challenges
 3. advanced progression
 4. premium flows
-

12. Zależności między etapami

12.1 Czego nie wolno robić za wcześnie

Nie należy rozpoczynać prac nad: - rozbudowanym dashboardem, - rankingami, - płatnościami, - rozbudowaną ścieżką lekcji, - animacjami high-fidelity,

zanim nie będzie działać pełna sesja realtime.

12.2 Krytyczne zależności

- scoring zależy od pitch detection i timing engine,
 - gameplay UI zależy od session runtime,
 - zapis wyników zależy od stabilnej struktury session result,
 - progres zależy od zapisanych wyników,
 - onboarding zależy od działającego engine detection flow.
-

13. Dokumenty robocze, które należy utworzyć równolegle

W trakcie realizacji należy prowadzić osobne dokumenty operacyjne:

13.1 Architecture Decision Record (ADR)

Każda istotna decyzja architektoniczna powinna zostać zapisana, np.: - dlaczego WebSocket, a nie HTTP polling, - dlaczego Rust, a nie inny stack, - dlaczego engine jest source of truth.

13.2 Protocol Specification

Dokładny opis: - typów wiadomości, - payloadów, - wersjonowania, - błędów, - heartbeat.

13.3 Data Model Specification

Opis encji, relacji i pól.

13.4 QA Test Plan

Scenariusze testowe dla: - połączenia z engine, - sesji treningowej, - błędów urządzeń, - zapisów do backendu.

13.5 Release Checklist

Lista warunków przed publikacją builda testowego.

14. Definition of Done dla kolejnych kamieni milowych

14.1 Milestone A — Engine PoC

Gotowe, gdy: - engine działa, - wykrywa urządzenia, - odbiera sygnał, - emituje nuty do webowego debug view.

14.2 Milestone B — Session Loop

Gotowe, gdy: - można uruchomić sesję, - można ocenić hit/miss, - można zakończyć sesję z wynikiem.

14.3 Milestone C — User Flow

Gotowe, gdy: - użytkownik loguje się, - wybiera trening, - przechodzi sesję, - zapisuje wynik, - widzi progres.

14.4 Milestone D — MVP Closed Alpha

Gotowe, gdy: - nowy użytkownik może sam przejść onboarding, - system działa stabilnie, - są logi i telemetria, - znane problemy są udokumentowane.

15. Checklista składania systemu w całość

To jest końcowa checklista integracyjna.

15.1 Engine

- uruchamia się lokalnie,
- ma wersję,
- udostępnia WebSocket,
- wykrywa urządzenia,
- potrafi rozpocząć sesję,
- emituje note eventy,
- liczy scoring,
- zwraca wynik końcowy.

15.2 Web

- wykrywa engine,
- potrafi połączyć się z WebSocket,
- potrafi pobrać konfigurację treningu,
- uruchamia ekran sesji,
- odbiera eventy,
- pokazuje feedback,
- zapisuje wynik.

15.3 Backend

- działa auth,
- działa content fetching,
- działa zapis sesji,
- działa aktualizacja progresu,
- działa pobieranie dashboardu.

15.4 UX

- użytkownik rozumie, co ma zrobić,
- komunikaty błędów są czytelne,
- brak martwych ekranów,
- onboarding jest zrozumiały.

15.5 Stabilność

- brak krytycznych crashy,
 - obsługowane podstawowe błędy urządzeń,
 - obsługowany brak engine,
 - obsługowana niezgodność wersji.
-

16. Najczęstsze błędy realizacyjne, których należy unikać

16.1 Budowanie zbyt dużej warstwy produktu przed działającym rdzeniem

To prowadzi do pięknego interfejsu bez działającej sesji realtime.

16.2 Zbyt szybkie dodawanie nowych trybów ćwiczeń

Najpierw musi powstać jeden stabilny loop.

16.3 Brak jawnego protokołu komunikacji

Jeżeli wiadomości web ↔ engine nie są formalnie opisane, integracja zacznie się rozpadać.

16.4 Liczenie scoringu po stronie web

To jest błąd architektoniczny i źródło przyszłych problemów.

16.5 Pomijanie diagnostyki i loggingu

Bez tego trudno będzie odróżnić błąd urządzenia od błędu algorytmu albo błędu połączenia.

17. Rekomendowany sposób pracy w VS Code + Codex

17.1 Strategia

Prace należy prowadzić modułowo, nie „całą aplikacją naraz”.

Najlepszy model: 1. jeden milestone, 2. jeden moduł, 3. jeden zdefiniowany rezultat, 4. test, 5. dopiero potem kolejny krok.

17.2 Dobra praktyka dla Codex

Dla każdego modułu należy przygotować: - cel modułu, - wejścia, - wyjścia, - ograniczenia, - definition of done, - przykładowe payloady, - kryteria testowe.

Dzięki temu Codex będzie generował bardziej spójny i użyteczny kod.

18. Podsumowanie wykonawcze

Najwłaściwsza kolejność realizacji projektu jest następująca:

1. ustalić architekturę i scope,
2. uruchomić local engine,
3. uruchomić audio capture,
4. wdrożyć onset + pitch detection,
5. wystawić lokalny WebSocket,
6. zbudować webowy debug client,
7. wdrożyć session runtime,
8. wdrożyć scoring,
9. zbudować ekran sesji,
10. zbudować ekran wyniku,
11. dołożyć backend, auth i zapis wyników,
12. dołożyć progres i dashboard,
13. dopracować stabilność integracji,
14. wdrożyć onboarding i kalibrację,
15. przygotować build do zamkniętych testów.

To jest właściwa droga dojścia do kompletnej, złożonej wersji programu.

Jeżeli ta kolejność zostanie zachowana, projekt będzie rozwijał się w sposób kontrolowany, a ryzyko utknięcia w pobocznych elementach znacząco spadnie.